A TECHNICAL SEMESTER PROJECT REPORT

COURSE: CS-302 | DATABASE MANAGEMENT SYSTEMS LABORATORY

CRICKET ANALYTICS SYSTEM

A 3NF Relational Database Architecture and Interactive Web Dashboard

Complete Database Documentation, Cardinality Analysis, Constraints & Procedural Triggers

---

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF TECHNOLOGY

IN COMPUTER SCIENCE AND ENGINEERING

CANDIDATE INFORMATION:

- Name: Ashmita Bhattacharya
- University Roll No: 2023-CS-114
- Academic Semester: V (3rd Year)
- Program: B.Tech Computer Science
- Track: Data & Database Systems

INSTITUTIONAL SUPERVISION:

- Dept: Computer Science & Engg.
- Faculty: Engineering & Technology
- Lab: Database Systems & Analytics
- Target Engine: PostgreSQL 16.x
- Academic Session: 2025 - 2026

CORE PLATFORM & FRAMEWORK SPECIFICATIONS

• Database Engine: PostgreSQL 16 • Backend: Python 3.11 • UI Framework: Streamlit • Data: Pandas & Plotly

Comprehensive Semester Report | Monochrome Edition | November 2026

1. Introduction and Project Overview

1.1 Domain Motivation and Context

• **Data Intensity of Cricket:** Every professional cricket match generates a cascade of discrete events (balls bowled, runs scored, extras, bowling dismissals, and over-by-over run rates).

• **Limitations of Conventional Media:** Traditional portals display match scores as pre-formatted, static scorecard texts, which prevents ad-hoc querying, historical trend exploration, or cross-entity analysis.

• **DBMS-Driven Solution:** This project applies formal relational database principles to structure cricket operations into a normalized, queryable schema, preserving ACID properties and referential integrity.

• **Integrated Analytics Portal:** A Python and Streamlit web application connects directly to PostgreSQL, allowing users to browse fixtures, query career dossiers, and inspect interactive visualizations.

2. Objectives and Scope

2.1 Core Engineering Objectives

• **3NF Schema Normalization:** Decompose cricket operations into nine normalized tables to eliminate update anomalies, data redundancy, and inconsistent recording.

• **Server-Side Business Rules:** Enforce domain constraints directly in PostgreSQL using declarative CHECK constraints, composite unique indexes, and four PL/pgSQL triggers.

• **Performant Query Abstraction:** Implement ten analytical SQL views and parameterized queries to derive batting averages, strike rates, and bowling economies with minimal application overhead.

• **Zero-JavaScript Web Dashboard:** Construct an intuitive, responsive Streamlit user interface featuring multi-entity search, interactive scorecards, and Plotly visualizations.

2.2 System Boundaries & User Personas

• **Target Beneficiaries:** Coaches assessing player form; analysts tracking team strategies; students examining real-world relational database architectures.

• **Project Scope:** Encompasses completed limited-overs fixtures (IPL 2024–2025 dataset) across 10 franchises, 60 players, 8 stadiums, and 135 discrete deliveries.

• **Boundary Constraints:** Excludes spatial sensor tracking (Hawk-Eye pitch coordinates and wagon wheel angles) due to the absence of public coordinate telemetry APIs.

3. Technology Stack

3.1 Technology Role and Selection Factors

Technology	System Role	Key Architectural Selection Factors
PostgreSQL 16	Relational Engine	ACID compliance, procedural PL/pgSQL triggers, foreign key cascade/restrict rules, cost-based planner.
Python 3.11	Application Backend	Standard DB-API client (psycopg 3), parameterized query safety, vectorized data manipulation.
Streamlit	Frontend Interface	Python-native reactive UI, eliminating JavaScript build overhead while maintaining session state routing.
Pandas 2.0	Tabular Structuring	Vectorized transformations, group-by aggregations, and seamless translation of SQL cursor records.
Plotly Express	Visual Analytics	Publication-grade interactive charts (bars, donuts, scatter plots) with responsive hover tooltips.

4. System Architecture and Data Flow

4.1 Tiered Operational Pipeline

• **Storage Tier (PostgreSQL):** Manages nine core tables, referential constraints, four business triggers, and ten analytical views.

• **Data Access Tier (Python/psycopg):** Executes parameterized SQL queries via connection pooling with a 60-second in-memory cache for fast UI response.

• **Presentation Tier (Streamlit):** Handles user input, manages session state routing, and dynamically renders metrics, scorecards, and Plotly charts.

5. Database Design: Entity Specifications

5.1 Relational Schema Dictionary (Nine Tables, ~515 Records)

Table Name	Primary Key	Foreign Keys	Attributes, Types & Integrity Constraints
TEAMS	team_id (INT)	None	team_name (VARCHAR(100), UQ, NOT NULL), country (VARCHAR(50)), coach
PLAYERS	player_id (INT)	team_id -> TEAMS	full_name, date_of_birth (DATE), role (CHECK: batter, bowler, all-rounder, wicketk
VENUES	venue_id (INT)	None	venue_name (VARCHAR(100)), city (VARCHAR(50)), country (VARCHAR(50)),
MATCHES	match_id (INT)	home_team_id, away_team_id, venue_id	match_date (DATE), match_format (CHECK: T20, ODI, Test), result_summary. CH
INNINGS	innings_id (INT)	match_id, batting_team_id	innings_number (CHECK: 1, 2), total_runs (>=0), wickets_lost (0..10), overs_played
BATTING_PERF	batting_id (INT)	innings_id, player_id	runs_scored (>=0), balls_faced (>=0), fours, sixes, dismissal, not_out (BOOL), battin
BOWLING_PERF	bowling_id (INT)	innings_id, player_id	overs_bowled (NUMERIC(3,1)), balls_bowled, maidens, runs_conceded (>=0), wick
FIELDING_PERF	fielding_id (INT)	innings_id, player_id	catches (SMALLINT), stumpings (SMALLINT), run_outs (SMALLINT). UQ: (innin
DELIVERIES	delivery_id (INT)	innings_id, striker_id, bowler_id	over_number (>=1), ball_number (1..6), runs_batter, extras, total_runs, wicket_type.

5.2 Key Strategy and Constraints

• **Surrogate Primary Keys:** Every table utilizes an auto-incrementing SERIAL integer primary key, optimizing B-tree index traversal and reducing foreign key storage footprint.

• **Natural Candidate Keys:** Enforced via composite UNIQUE constraints: (match_id, innings_number) in INNINGS; (innings_id, player_id) in performance tables; and (innings_id, over_number, ball_number) in DELIVERIES.

• **Domain Integrity Checks:** CHECK constraints guarantee realistic boundaries: capacity > 0; home_team_id <> away_team_id; innings_number IN (1, 2); and striker_id <> bowler_id.

6. Normalization and Integrity Constraints

6.1 Formal Normalization Analysis (1NF to 3NF)

• **First Normal Form (1NF) Compliance:** All attributes are atomic and indivisible. Multi-valued elements (such as multiple overs bowled by a player or multiple boundaries) are stored as separate tuples in normalized tables, avoiding comma-separated strings or nested arrays.

• **Second Normal Form (2NF) Compliance:** The schema is in 1NF and contains zero partial functional dependencies. In composite candidate keys like (innings_id, player_id), every non-key attribute (e.g. runs_scored, overs_bowled) depends functionally on the entire composite key, not on any single component.

• **Third Normal Form (3NF) Compliance:** The schema is in 2NF and contains zero transitive functional dependencies (X -> Y -> Z). Player biographical attributes (full_name, date_of_birth, role, styles) depend exclusively on player_id in PLAYERS; performance records reference only player_id, eliminating redundant personal data across matches.

6.2 Relational Integrity Constraints

• **Entity Integrity:** Every relation defines a NOT NULL, unique PRIMARY KEY enforced via an underlying B-tree index, guaranteeing that every row is distinctly identifiable.

• **Referential Integrity:** Foreign key constraints establish valid associations across parent and child relations, backed by strict deletion policies:

– **ON DELETE RESTRICT Policy:** Applied to master entities (TEAMS, VENUES) referenced by MATCHES. The engine prevents dropping a stadium or franchise that is associated with recorded fixtures.

– **ON DELETE CASCADE Policy:** Applied to hierarchical telemetry tables (INNINGS, DELIVERIES, BATTING_PERFORMANCE, BOWLING_PERFORMANCE, FIELDING_PERFORMANCE). Deleting a fixture cleanly purges all child innings and deliveries atomically.

– **ON DELETE SET NULL Policy:** Configured on PLAYERS.team_id, permitting an athlete to become a free agent or change rosters without deleting their historical performance telemetry.

• **Domain Constraints & CHECK Rules:** Engine-level rules validate allowable data values:

– **Stadium Capacity Validation:** capacity > 0 ensures stadium spectator counts are strictly positive.

– **Fixture Integrity:** CHECK (home_team_id <> away_team_id) prevents a franchise from scheduling a fixture against itself.

– **Delivery Participants:** CHECK (striker_id <> bowler_id) ensures the batter facing a ball cannot simultaneously be the bowler delivering it.

7. Relationships, Cardinality, and Participation Constraints

7.1 Formal Definitions of Cardinality and Participation

• **Mapping Cardinality:** Defines the maximum number of entity instances in one set that can be associated with entity instances in another set (1:1, 1:N, or M:N).

• **Participation Constraints:** Specifies the minimum number of entity instances required in an association:

– **Total Participation (Mandatory, 1..N / 1..1):** Every instance in the entity set must participate in the relationship. Denoted by a double line or mandatory crossbar.

– **Partial Participation (Optional, 0..N / 0..1):** Instances may exist independently without participating in the relationship. Denoted by a single line or open circle.

7.2 Comprehensive Relationship & Participation Matrix

Relationship Nar	Source	Target	Card.	Source Part.	Target Part.	Foreign Key & Semantic Rules
Hosted At	VENUES	MATCHES	1 : N	Partial (0..N)	Total (1..1)	MATCHES.venue_id (Venue may host 0 matches; match must ha
Contracts	TEAMS	PLAYERS	1 : N	Partial (0..N)	Partial (0..1)	PLAYERS.team_id (Team has 0..N players; player may be unass
Home Team	TEAMS	MATCHES	1 : N	Partial (0..N)	Total (1..1)	MATCHES.home_team_id (Team plays 0..N home games; match
Away Team	TEAMS	MATCHES	1 : N	Partial (0..N)	Total (1..1)	MATCHES.away_team_id (Team plays 0..N away games; match
Innings Of	MATCHES	INNINGS	1 : 2	Total (2..2)	Total (1..1)	INNINGS.match_id (Completed match has exactly 2 innings; inn
Bats In	TEAMS	INNINGS	1 : N	Partial (0..N)	Total (1..1)	INNINGS.batting_team_id (Team bats in 0..N innings; innings m
Batting Perf.	INNINGS	PLAYERS	M : N	Total (1..11)	Partial (0..N)	Via BATTING_PERFORMANCE (innings has 1..11 batters; play
Bowling Perf.	INNINGS	PLAYERS	M : N	Total (1..N)	Partial (0..N)	Via BOWLING_PERFORMANCE (innings has 1..N bowlers; pla
Fielding Perf.	INNINGS	PLAYERS	M : N	Partial (0..N)	Partial (0..N)	Via FIELDING_PERFORMANCE (innings has 0..N catchers; pla
Ball Telemetry	INNINGS	DELIVERIES	1 : N	Partial (0..120)	Total (1..1)	DELIVERIES.innings_id (Innings has up to 120 balls; delivery b
Ball Striker	PLAYERS	DELIVERIES	1 : N	Partial (0..N)	Total (1..1)	DELIVERIES.striker_id (Player faces 0..N deliveries; delivery m
Ball Bowler	PLAYERS	DELIVERIES	1 : N	Partial (0..N)	Total (1..1)	DELIVERIES.bowler_id (Player delivers 0..N balls; delivery mus

7.3 Resolution of Many-to-Many Relationships

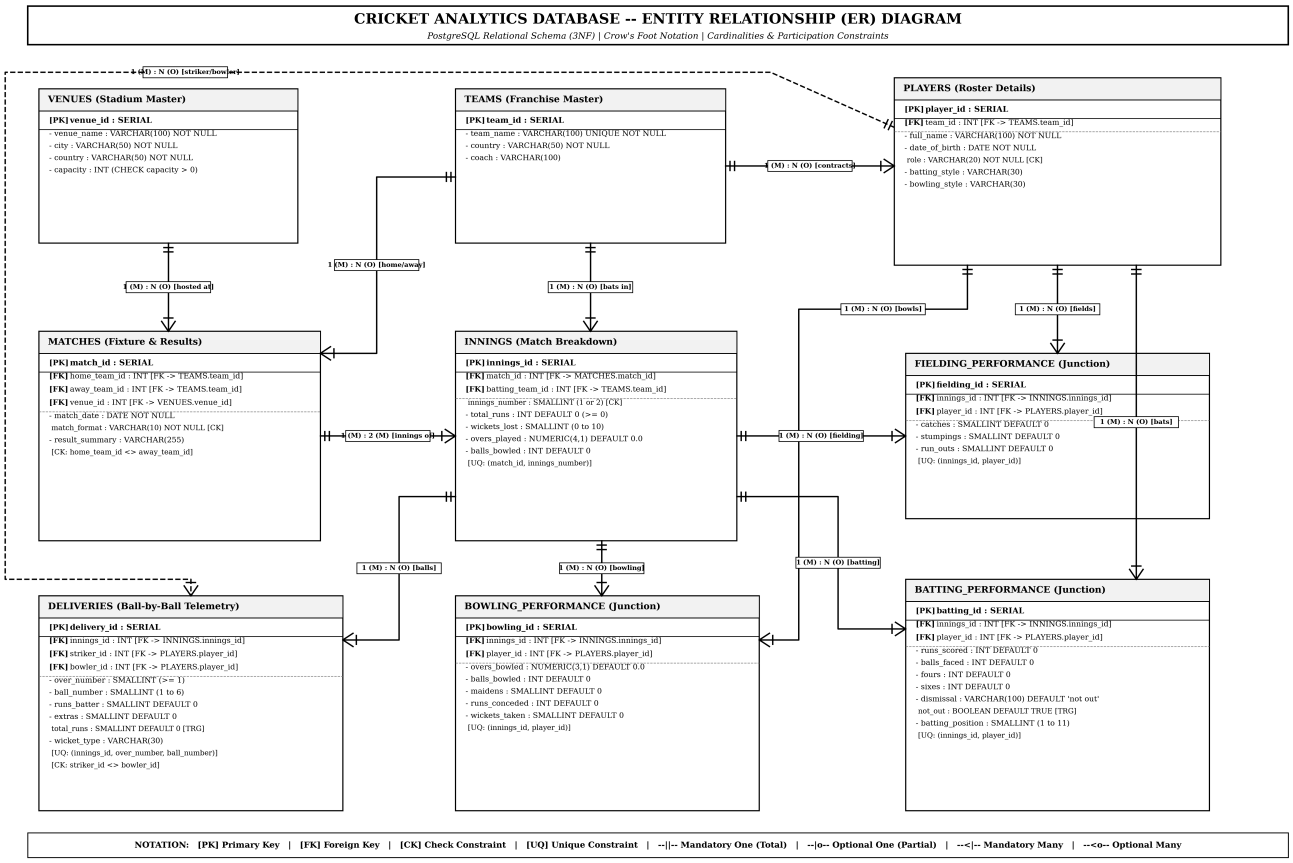
• **The M:N Association Problem:** Players participate in multiple matches, and each match involves 22 athletes. Directly associating PLAYERS and INNINGS would produce redundant multi-valued tuples and sparse null fields.

• **Decomposition into Three Junction Tables:** The M:N relationship is resolved into three specialized entities: BATTING_PERFORMANCE, BOWLING_PERFORMANCE, and FIELDING_PERFORMANCE. Each records distinct metrics (runs, overs, catches) with its own surrogate PK and composite candidate key.

8. Entity Relationship (ER) Diagram

8.1 Diagrammatic Representation (Crow's Foot Notation)

• **Formal ER Architecture:** The diagram below illustrates the nine relational tables, primary keys, foreign keys, constraints, and cardinalities in standard monochrome Crow's Foot notation.



8.2 Structural Walkthrough of Diagram Connectors

• **VENUE to MATCHES Connector:** Mandatory One crossbars at VENUES (every match requires exactly one venue); Optional Many fork at MATCHES (a venue can exist before hosting matches).

• **TEAM to MATCHES Dual Connectors:** Two independent 1:N relations (home_team_id, away_team_id). Both are Total for MATCHES, ensuring valid participant franchises for every fixture.

• **MATCHES to INNINGS Composition:** Mandatory 1:2 relationship. Exactly two sequential innings exist for every completed limited-overs match, governed by UNIQUE (match_id, innings_number).

• **Performance Junction Connectors:** INNINGS and PLAYERS connect to BATTING_PERFORMANCE, BOWLING_PERFORMANCE, and FIELDING_PERFORMANCE through 1:N links, ensuring independent recording of distinct athletic disciplines.

9. Procedural Logic & Analytical Abstractions

9.1 PL/pgSQL Domain Triggers

• **Trigger 1: trg_set_delivery_total:** Runs BEFORE INSERT OR UPDATE on DELIVERIES. Computes NEW.total_runs := COALESCE(NEW.runs_batter, 0) + COALESCE(NEW.extras, 0). Eliminates run arithmetic discrepancies.

• **Trigger 2: trg_validate_overs_format:** Runs BEFORE INSERT OR UPDATE on BOWLING_PERFORMANCE. Validates that the decimal part of overs_bowled is between .0 and .5. Throws an exception if .6 through .9 is entered, directly enforcing the 6-balls-per-over rule.

• **Trigger 3: trg_sync_batting_not_out:** Runs BEFORE INSERT OR UPDATE on BATTING_PERFORMANCE. Automatically forces not_out = TRUE if dismissal is 'not out', and toggles not_out = FALSE when a dismissal method is recorded.

• **Trigger 4: trg_validate_player_age:** Runs BEFORE INSERT OR UPDATE on PLAYERS. Enforces date_of_birth <= CURRENT_DATE - INTERVAL '15 years', preventing the registration of invalid dates or underage athletes.

9.2 Pre-Aggregated Relational Views

• **TOP_RUN_SCORERS View:** Aggregates career innings, runs, highest scores, centuries, fifties, batting strike rates, and batting averages.

• **TOP_WICKET_TAKERS View:** Calculates total wickets, overs bowled, maidens, runs conceded, bowling economy rates, and bowling averages.

• **TEAM_PERFORMANCE View:** Computes tournament standings: matches played, wins, losses, win percentage, and total runs scored/conceded.

• **MATCH_SCORECARD View:** Provides unified match sheets joining teams, venues, innings totals, and match results.

• **VENUE_STATISTICS View:** Analyzes venue utilization, average first-innings totals, and toss-winner match outcomes.

10. Application Implementation and Query Processing

10.1 Data Access Layer and Parameterization

• **Parameterized Queries:** Database operations in database.py utilize psycopg 3 with %s placeholder bindings, preventing SQL injection vulnerabilities.

• **In-Memory Caching:** Queries are cached with Streamlit's @st.cache_data(ttl=60) decorator, providing snappy UI transitions while reflecting database updates within 60 seconds.

• **Deferred Navigation State Pattern:** Redirects between search chips, leaderboards, and profile pages via st.session_state target variables, eliminating widget key collisions.

11. Representative SQL Query Operations

11.1 Dynamic Batting Aggregate Query (from queries.py)

Listing: Career Batting Aggregation with Dynamic Strike Rate

```
 SELECT p.player_id, p.full_name, t.team_name,
       COUNT(bp.batting_id) AS innings_played,
       SUM(bp.runs_scored) AS total_runs,
       MAX(bp.runs_scored) AS highest_score,
       ROUND(SUM(bp.runs_scored)::DECIMAL / NULLIF(SUM(CASE WHEN bp.not_out = FALSE THEN 1 ELSE 0 END), 0), 2) AS batting_avg,
       ROUND((SUM(bp.runs_scored)::DECIMAL / NULLIF(SUM(bp.balls_faced), 0)) * 100, 2) AS strike_rate
FROM PLAYERS p
JOIN TEAMS t ON p.team_id = t.team_id
JOIN BATTING_PERFORMANCE bp ON p.player_id = bp.player_id
GROUP BY p.player_id, p.full_name, t.team_name
ORDER BY total_runs DESC LIMIT 10;
```

11.2 Innings Bowling Figures Query

Listing: Innings Bowling Extraction with Parameterized Filter

```
 SELECT bp.player_id, p.full_name, bp.overs_bowled, bp.maidens, bp.runs_conceded, bp.wickets_taken,
       ROUND((bp.runs_conceded::DECIMAL / NULLIF(bp.overs_bowled, 0)), 2) AS economy_rate
FROM BOWLING_PERFORMANCE bp
JOIN PLAYERS p ON bp.player_id = p.player_id
WHERE bp.innings_id = %s
ORDER BY bp.wickets_taken DESC, economy_rate ASC;
```

12. Conclusion and Future Enhancements

12.1 Project Accomplishments & Technical Competencies

• **Relational Modeling Rigor:** Demonstrated end-to-end 3NF decomposition, eliminating update anomalies while preserving domain consistency.

• **Server-Side Automation:** Enforced business rules using declarative constraints and procedural PL/pgSQL triggers inside PostgreSQL.

• **Full-Stack Python Synergy:** Delivered a responsive web interface with zero JavaScript overhead, integrating psycopg, Pandas, and Plotly.

12.2 Realistic Future Enhancements

• **Automated Ingestion Pipeline:** Build an automated ETL worker to ingest ball-by-ball JSON feeds directly from open-source repositories.

• **Predictive Machine Learning:** Integrate logistic regression models to compute live in-play win probabilities based on required run rates.

• **Spatial Telemetry Mapping:** Augment DELIVERIES with pitch landing coordinates (X, Y) and wagon wheel polar angles.

ACADEMIC VERIFICATION & SIGN-OFF

• Verification: This project report and associated relational database represent authentic, verified student coursework.

• Database Engine: PostgreSQL 16 | Font: Times New Roman | Monochrome Edition (Bullet Structured)